

Course	Offensive Security Operations II
Name	Janet Oluwatosin Michael
Legacy Course Code	CIP-A105
Student id	2026/EHIT/17073
Assessment	CTF 1 - Summative Competency Exam
Target Designation	OPFOR-01
Target Profile	Standalone intentionally vulnerable Linux server
Assessment Window	06 Sep 2026 00:00 WAT - 16 Sep 2026 23:59 WAT

Executive Report

Assessment Overview

Operation Iron Raven was a practical offensive security assessment carried out on the Academy-provided OPFOR-01 Linux machine. The aim was to identify the services running on the machine, check the web applications for possible vulnerabilities, test some of the vulnerabilities found, and see if I could gain access to the system.

The target IP address was **192.168.56.104** and the hostname was **raven.local**. My Kali machine was **192.168.56.105**. The testing was done inside the isolated lab environment provided for the assessment.

Overall Risk

During the assessment, I found some security issues that could make the server easier to attack.

The main things I found were:

- Port 22 was open for SSH.
- Port 80 was running Apache 2.4.10.
- WordPress 4.8.7 was running on the server.
- The WordPress REST API exposed the username **michael** without authentication.
- The /vendor/ directory was publicly accessible and directory listing was enabled.
- The vendor directory exposed PHPMailer **5.2.16** and other dependency files.
- The contact form accepted invalid and unusual email input without showing an error.
- I also tested some possible WordPress and PHPMailer vulnerabilities.

These findings do not mean that I successfully took over the machine. I did not get a working shell or root access during the assessment.

Attack Summary

I started with network discovery and Nmap scanning. I found three main open services: SSH on port 22, HTTP on port 80 and RPCbind on port 111. I focused mainly on the HTTP service because it exposed a website and a WordPress installation.

From the WordPress REST API, I was able to identify the username **michael** without logging in. I also confirmed that the WordPress version was **4.8.7**. While checking the website, I found the `/vendor/` directory. Directory listing was enabled, so I could see the files inside it. One of the files showed that the server was using **PHPMailer 5.2.16**.

Because this version was associated with a known PHPMailer vulnerability, I investigated it as one of my main attack possibilities. I also tested the contact form because it accepted email input from the user.

I tested different types of input on the contact form, including an invalid email, a malformed email and a long email address. The application still returned **“Message has been sent.”** However, I could not confirm exactly how the mail function was being handled from the outside. I then attempted the PHPMailer attack using the available Metasploit module. The module tried to write a PHP payload to the target, but no session was created. When I checked the generated PHP file through the browser, it returned **404 Not Found**.

I also investigated another WordPress vulnerability, but I could not demonstrate the vulnerability on this target. RPCbind was also checked, but I did not find a useful file-sharing service through it.

Business Impact

If this type of setup was used on a real system, the information exposed could help an attacker understand the server better and plan further attacks. For example, exposing usernames can help with account attacks, while exposing software versions and dependency files can help an attacker look for known vulnerabilities.

The `/vendor/` directory is also a concern because dependency files and version information were available without authentication. However, based on my testing, I did not obtain evidence of a complete compromise. I did not get an interactive shell, root access or any confirmed privileged access.

Priority Actions

Based on what I found, I would recommend the following actions:

1. Remove the public `/vendor/` directory listing and prevent users from accessing dependency files directly.
2. Update the outdated software and dependencies, especially the versions of Apache, WordPress and PHPMailer identified during the assessment.
3. Review the contact form, especially how the email input is processed on the server.

4. Reduce unnecessary information disclosure from the WordPress REST API, including public exposure of usernames.
5. Review exposed services such as SSH and RPCbind and disable any service that is not required.

Scope and Rules of Engagement

This assessment was carried out as part of the Academy's Operation Iron Raven CTF. The target provided for the assessment was the OPFOR-01 Linux machine.

Target

- **Target IP:** 192.168.56.104
- **Hostname:** raven.local
- **Attacker machine:** Kali Linux
- **Kali IP:** 192.168.56.105

The testing was done inside the isolated lab environment using the Host-Only network in VirtualBox. I only carried out testing against the assigned OPFOR-01 target.

Scope

The assessment included:

- Network and service discovery
- Port and service enumeration
- Web application enumeration
- WordPress enumeration
- Vulnerability identification and analysis
- Controlled vulnerability testing
- Initial-access attempts
- Post-exploitation and privilege-escalation investigation where access was available
- Collection of evidence from the tests carried out
- The main services investigated were the HTTP, SSH and RPCbind services found during the Nmap scan.

Rules Followed

- I stayed within the assigned target and the isolated assessment network. I did not intentionally scan or attack other systems outside the OPFOR-01 machine. I did not carry out denial-of-service attacks, resource exhaustion or uncontrolled fuzzing. I also tried to keep evidence of the commands I ran, the results I received and the decisions I made during the assessment.

Where a possible vulnerability was identified, I treated it as a possible attack path until I had enough evidence to confirm it. For example, I identified PHPMailer 5.2.16 and

investigated a related vulnerability, but I did not report successful exploitation because my test did not create a session. The purpose of the testing was to identify and demonstrate security weaknesses within the authorized lab environment while keeping the testing controlled.

4. Laboratory Architecture and Methodology

The assessment was carried out using a Kali Linux virtual machine and the Academy-provided OPFOR-01 virtual machine in VirtualBox. Both machines were connected through a **Host-Only network**, so the testing was kept inside the lab environment.

The main IP addresses used during the assessment were:

- **Kali Linux:** 192.168.56.105
- **OPFOR-01:** 192.168.56.104
- **Target hostname:** raven.local

I used Kali Linux for the scanning, enumeration and vulnerability testing.

4.2 Methodology

I started by identifying the target and checking the services that were available. I used Nmap to find open ports and identify the versions of the services running on the machine.

The main scan identified:

- SSH on port 22
- HTTP on port 80
- RPCbind on port 111

After identifying the HTTP service, I focused more on the web application because it exposed both a custom website and a WordPress installation. I used curl to manually inspect the website, WordPress pages, REST API responses and other files. I also used Gobuster to check some directories and SearchSploit and Metasploit to investigate possible vulnerabilities.

For WordPress, I checked the REST API, users, posts, pages and plugin-related paths. This allowed me to identify the WordPress version and the username michael. During the web enumeration, I also found the /vendor/ directory. The directory listing exposed dependency files, and checking the VERSION file showed PHPMailer 5.2.16. I then investigated whether the exposed PHPMailer version could provide an attack path. I looked at the available exploit information and documented my reasoning before attempting exploitation. I also tested the contact form with different types of input to understand how it handled user-controlled email values.

For the PHPMailer attack, I used the available Metasploit module and recorded the result. The module attempted to write a PHP payload, but no session was created. I then checked the

generated file through HTTP and received a 404 response. I also investigated the SSH and RPCbind services. The RPCbind service was checked with the rpcinfo Nmap script, but I did not find a useful file-sharing service. I also did not obtain SSH access. Throughout the assessment, I saved important command outputs and test results in the evidence directory so that I could use them later when writing the report.

4.3 Evidence Collection

I kept evidence for important parts of the assessment, including:

- Nmap service scan
- WordPress REST API response
- WordPress username enumeration
- Vendor directory information
- PHPMailer version
- Composer information
- RPC enumeration
- Contact form tests
- WordPress vulnerability test
- PHPMailer exploitation attempt
- HTTP response from the attempted payload

This helped me make sure that the findings in this report were based on things I actually observed during the assessment.

Tools used

1. **Nmap** — host/service and version enumeration
2. **curl** — manual web and HTTP testing
3. **Gobuster** — directory enumeration
4. **jq** — parsing WordPress REST API JSON
5. **SearchSploit** — searching and reviewing known exploits
6. **Metasploit Framework** — vulnerability/exploitation testing
7. **rpcinfo Nmap script** — RPC service enumeration
8. **grep / sed / head** — inspecting responses and extracting useful information
9. **Browser** — manually checking the website and generated URL
10. **VirtualBox** — laboratory environment/network setup
11. **Kali Linux** — attacker workstation.

5. Attack-Surface Summary

5.1 Network Services

5.1 Network Services

The main Nmap scan was:

```
sudo nmap -sV -O 192.168.56.104 -oA ~/iron-raven/evidence/target-services
```

The scan identified:

Port	Service	Version
22/tcp	SSH	OpenSSH 6.7p1
80/tcp	HTTP	Apache 2.4.10
111/tcp	RPCbind	RPCbind 2-4

The HTTP service became my main focus because it exposed several web applications and files.

5.2 Website

The target had a custom website with pages and resources such as:

- index.html
- about.html
- service.html
- team.html
- contact.php
- blog-single.html
- JavaScript and CSS files

The website also contained a WordPress installation under:

/wordpress/

5.3 WordPress

The WordPress installation was identified as **version 4.8.7**.

The WordPress REST API was accessible without authentication.

The API exposed routes for users, posts, pages, comments, media and other WordPress functions.

The users endpoint disclosed the username: michael

I also checked the available public posts and pages. I did not find private content that could be used to confirm a successful WordPress attack.

5.4 Vendor Directory

The following directory was publicly accessible:

/vendor/

Directory listing was enabled.

The directory contained files including:

VERSION
composer.json
composer.lock
class.phpmailer.php
class.smtp.php
PHPMailerAutoload.php

The VERSION file showed: 5.2.16

This identified the exposed PHPMailer version.

5.5 Contact Form

The custom website had a contact form at:

/contact.php

The form accepted:

- name
- email
- subject
- message

I tested normal and malformed values through the form. The application returned HTTP 200 and: “**Message has been sent.**” And I did not see an error from PHPMailer or SMTP in the responses.

5.6 RPCbind

Port 111 was investigated using the Nmap RPC information script.

The results showed RPCbind and status services, but I did not identify an NFS or other useful file-sharing service. Because of this, RPCbind was not pursued further as a main attack path.

6. Confirmed Findings and Risk Ratings

I separated the findings that I actually confirmed from vulnerabilities that I only investigated.

Finding 1 – WordPress Username Disclosure

Severity: Medium

Affected asset: WordPress REST API

Evidence: wp-users.txt and wp-user-1.json

The WordPress REST API allowed an unauthenticated request to return the username **michael**.

This is an information disclosure issue because a valid WordPress username was available without authentication.

Impact

An attacker could use the username as information for further account-targeting attacks.

Recommendation

Review the WordPress REST API and reduce unnecessary unauthenticated user information where possible. Strong passwords and appropriate login protections should also be used.

Finding 2 – Public /vendor/ Directory Listing

Severity: Medium

Affected asset: http://raven.local/vendor/

Evidence: /vendor/ directory response

The /vendor/ directory displayed its contents to an unauthenticated visitor.

This exposed dependency and support files that should normally not be publicly accessible.

Impact

The information could help an attacker identify technologies and versions being used by the application.

Recommendation

Disable directory listing and prevent dependency files from being directly accessed through the web server. Where possible, the vendor directory should not be inside the public web root.

Finding 3 – Exposed PHPMailer Version

Severity: Medium

Affected asset: /vendor/VERSION

Evidence: vendor-version.txt

The exposed VERSION file showed PHPMailer **5.2.16**.

This was important because the version falls within the historical affected range of a known PHPMailer vulnerability.

Impact

An attacker can use exposed version information to research vulnerabilities that may apply to the application.

Recommendation

Update PHPMailer to a supported version and avoid exposing dependency version information publicly.

Finding 4 – Contact Form Input Handling

Severity: Low / Medium

Affected asset: /contact.php

Evidence:

- contact-invalid-email.txt
- contact-malformed-email.txt
- contact-long-email-validation.txt

I tested invalid, malformed and long email values. The application continued to return HTTP 200 and: “**Message has been sent.**” This shows that the input handling should be reviewed.

However, I did **not** prove that this resulted in code execution, command injection or another direct compromise. Therefore, I am not claiming those vulnerabilities based only on these tests.

Recommendation

The application should properly validate email input and review how user-controlled values are passed to the mail-sending function.

7. Initial-Access Narrative

After identifying PHPMailer 5.2.16, I investigated whether it could be used to gain initial access.

I found Exploit-DB entry **40974**, which relates to a PHPMailer argument-injection vulnerability affecting versions below 5.2.18. I inspected the exploit and also checked the available Metasploit module. The reason I considered this a possible attack path was because:

- PHPMailer 5.2.16 was exposed.
- The version was within the affected version range.
- The contact form accepted attacker-controlled email input.

However, one thing I could not confirm was whether the contact.php application was actually using the exposed PHPMailer installation. I documented this as an attack hypothesis instead of immediately treating it as a confirmed vulnerability. I then used the Metasploit PHPMailer argument-injection module for a controlled exploitation attempt.

The module was configured against:

RHOSTS: 192.168.56.104
RPORT: 80
LHOST: 192.168.56.105

TARGETURI: /The module reported that it attempted to write a PHP payload:
/var/www/2p2hOS7T.php

It then waited for the payload to be triggered.

The final result was: “ Exploit completed, but no session was created.”

I then checked the generated PHP path through HTTP. The server returned: “404 Not Found”

Therefore, I could not confirm that the payload was successfully written to the web-accessible directory or executed. I did not claim this as successful initial access

8. Post-Exploitation and Privilege Escalation

I did not obtain an interactive shell during the assessment.

Because there was no confirmed shell, I could not carry out normal post-exploitation activities such as:

- identifying the current user
- checking local processes
- checking running services from inside the target
- looking for credentials
- checking file permissions
- investigating local configuration files
- testing local privilege escalation

I therefore did not obtain or claim root access. The assessment did not produce evidence of successful privilege escalation. This was one of the areas where I was not able to complete the full attack chain.

9. Mission Objectives and Proof of Completion

The main objective of the assessment was to investigate the assigned target and progress through reconnaissance, vulnerability analysis, initial access and post-exploitation.

I completed the reconnaissance and enumeration stages. I identified the main network services and investigated the web application. I also identified confirmed information disclosure issues and investigated the PHPMailer attack path. However, I did not complete the full compromise chain.

Completed

- Target identification
- Network/service enumeration
- Web application enumeration
- WordPress enumeration
- Attack-surface analysis
- Vulnerability analysis
- Controlled exploitation attempt
- Evidence collection

Not completed

- Confirmed interactive shell
- Post-exploitation
- Privilege escalation
- Root access
- Mission/flag collection

I have not claimed any flag or objective that I could not verify with evidence.

10. Remediation Roadmap

Immediate Actions

The first actions should be to remove the most obvious information exposure.

- Disable directory listing on /vendor/.
- Prevent direct access to dependency files.
- Review the WordPress REST API user information.
- Review the contact form input validation.
- Update the exposed PHPMailer version.

Short-Term Actions

The server should be reviewed for outdated software and unnecessary services.

- Update the WordPress installation and its components.
- Update Apache and other outdated software.
- Review SSH access and restrict it to authorized users.
- Disable RPCbind if it is not required.
- Review the mail functionality used by contact.php.

Strategic Actions

For longer-term security, the organization should have a process for:

- regular vulnerability scanning
- keeping software and dependencies updated
- reviewing public web directories
- monitoring exposed services
- secure configuration of web applications
- periodic penetration testing
- reviewing and validating security controls after remediation

After the fixes are made, the identified findings should be retested to confirm that they have been properly resolved.

11. Conclusion and Lessons Learned

This assessment helped me understand the difference between finding a possible vulnerability and actually proving that it can be exploited.

One of the main examples was PHPMailer. I found PHPMailer 5.2.16 exposed on the target and identified a known vulnerability that appeared to apply to that version. I then attempted exploitation, but the attempt did not create a session and the generated PHP file returned 404 when I checked it. This showed me that a vulnerable version by itself is not enough to claim successful exploitation. I also needed to understand how the application was using the vulnerable component and whether the required conditions for exploitation were actually present.

I also learned the importance of keeping evidence while working. I saved Nmap results, WordPress REST API information, vendor information, contact form tests, RPC results and the Metasploit output. This made it easier to separate what I confirmed from what I only suspected. My main technical gap from this assessment was initial access and what comes after initial access. I was able to perform reconnaissance, enumeration and vulnerability analysis, but I did not get a working shell, so I could not complete post-exploitation or privilege escalation.

For my next operation, I need to improve my understanding of validating exploitation conditions, initial-access techniques, Linux post-exploitation and privilege-escalation techniques. Overall, I was able to complete a good part of the assessment methodology and identify real security issues on the target, but I did not complete the full compromise chain.

APPENDIX A — OPERATOR ACTIVITY LOG

Timestamp Note

I did not record the exact time for every command during the assessment. However, some of the saved evidence contained timestamps. I have included those timestamps where they were available and left the other entries as **Not recorded** instead of guessing

The activities below are arranged in the order they were performed.

Date / Time	Phase	Action / Command Purpose	Key Observation / Result	Decision / Outcome
13 Sep 2026, 12:39 - 0400 / 16:39 WAT	Phase 1	Network discovery and host identification	Hosts discovered included 192.168.56.104, which was identified as the OPFOR-01 target	Target selected for further enumeration
15 Sep 2026, 11:53 - 0400 / 15:53 WAT	Phase 2	Nmap service and version enumeration against 192.168.56.104	Ports 22, 80 and 111 were open. SSH, Apache HTTP and RPCbind were identified	HTTP became the main area of investigation
Not recorded	Phase 2	HTTP enumeration using curl	Apache 2.4.10 was identified and /wordpress/ was found	Continued with WordPress enumeration
Not recorded	Phase 2	WordPress REST API enumeration	WordPress 4.8.7 and several REST API routes were identified	Investigated the users endpoint
Not recorded	Phase 2	WordPress user enumeration	Unauthenticated request disclosed the username michael	Recorded as an information-disclosure finding
Not recorded	Phase 2	WordPress posts/pages enumeration	Only normal public content was observed	No private-content exposure was claimed
Not recorded	Phase 2	Plugin directory enumeration	No useful plugin directory was identified using the available wordlist	Plugin-specific exploitation was not pursued

Not recorded	Phase 2	Root directory enumeration with Gobuster	contact.php, /vendor/, /wordpress/, /manual/ and other resources were identified	Investigated the exposed directories and applications
Not recorded	Phase 2	/vendor/ inspection	Directory listing was enabled and dependency files were publicly accessible	Investigated the exposed dependency information
Not recorded	Phase 2	PHPMailer version check	/vendor/VERSION returned 5.2.16	PHPMailer became a possible vulnerability hypothesis
Not recorded	Phase 2	Composer metadata inspection	composer.json and composer.lock were publicly readable	Recorded dependency information exposure
Not recorded	Phase 2	RPC enumeration with Nmap rpcinfo	RPCbind and status services were identified, but no NFS/file-sharing service was found	RPCbind was deprioritized
Not recorded	Phase 3	PHPMailer vulnerability analysis	PHPMailer 5.2.16 falls within the historical affected version range for the investigated CVE	PHPMailer was selected as an attack hypothesis, not a confirmed vulnerability
Not recorded	Phase 3	Attack hypothesis documented	PHPMailer exposure and the contact form were documented, but the connection between them was not confirmed	Proceeded with controlled validation
Not recorded	Phase 3	Contact form baseline POST	Application returned HTTP 200 and Message has been sent.	Confirmed that the form processed submitted data
Not recorded	Phase 3	Invalid email test	Invalid email value was accepted and the same success message was returned	Continued controlled input testing

Not recorded	Phase 3	HTTP header and application checks	Apache and WordPress information was exposed through responses and headers	Information was recorded for the report
Not recorded	Phase 4	SearchSploit / Exploit-DB investigation of PHPMailer	Exploit-DB 40974 was identified for PHPMailer versions below 5.2.18	Investigated whether the target conditions matched
Not recorded	Phase 4	Metasploit PHPMailer module configuration	Module was configured against 192.168.56.104; target version selected was PHPMailer <5.2.18	Controlled exploitation attempt was made
Not recorded	Phase 4	PHPMailer exploitation attempt	Metasploit reported that it attempted to write a payload to /var/www/2p2hOS7T.php but no session was created	Exploitation was unsuccessful
Not recorded	Phase 4	HTTP check for generated PHP file	Request to /2p2hOS7T.php returned HTTP 404	Initial access was not confirmed; possible web-root/path mismatch was noted

APPENDIX B — RISK REGISTER

Finding 1 — Publicly Accessible /vendor/ Directory

Asset: Web server — Apache 2.4.10

Finding: The /vendor/ directory was publicly accessible and directory listing was enabled. This allowed files and dependency information to be viewed without authentication.

Evidence: The directory listing showed files including composer.json, composer.lock, VERSION, PHPMailer files and other dependency-related files.

Likelihood: High, because the directory could be accessed directly through the web server without authentication.

Impact: Medium. An attacker can learn information about software and dependencies running on the server. This information could be used to identify possible vulnerabilities

Severity: Medium.

Remediation: Disable directory listing and prevent public access to the vendor/dependency directory. Web application dependencies should be stored outside the public web root where possible.

Finding 2 — WordPress Username Disclosure

Asset: WordPress 4.8.7

Finding: The WordPress REST API allowed an unauthenticated request to obtain user information. The username michael was disclosed.

Evidence: The request to the WordPress REST API users endpoint returned the user information and the username michael. This was saved in wp-users.txt and wp-user-1.json.

Likelihood: High, because the information was available without authentication.

Impact: Low to Medium. Username disclosure does not provide access by itself, but it gives an attacker useful information for further attacks against the login page.

Severity: Medium.

Remediation: Review whether public user enumeration is required. Restrict unnecessary user information from unauthenticated API requests and apply appropriate access controls.

Finding 3 — Outdated PHPMailer Dependency Exposed

Asset: Web server /vendor/ directory

Finding: The exposed /vendor/VERSION file identified PHPMailer version **5.2.16**.

Evidence: The request to /vendor/VERSION returned 5.2.16, which was saved as vendor-version.txt.

Likelihood: Medium to High because the version information was publicly accessible.

Impact: Medium. Knowing the exact dependency version can help an attacker research known vulnerabilities.

Severity: Medium.

Important Note: I did not confirm that the contact form was actually using this exposed PHPMailer installation. I also did not confirm successful remote code execution. Therefore, this finding is treated as exposed/outdated dependency information and not as a confirmed PHPMailer RCE.

Remediation: Update PHPMailer and other third-party dependencies to supported versions. Remove dependency version information from public access and review which libraries are actually being used by the application.

Finding 4 — Permissive Contact Form Input Handling

Asset: contact.php

Finding: The contact form accepted invalid, malformed and long email input and still returned Message has been sent.

Evidence: I tested invalid, malformed and long email values. The server returned HTTP 200 and the success message for the tests. The results were saved in the contact-form evidence files.

Likelihood: Medium.

Impact: Low to Medium. Weak input validation could become a problem if the submitted values are passed to other server-side components without proper validation.

Severity: Low/Medium.

Important Note: The testing did not prove code execution, command injection or successful exploitation through the contact form.

Remediation: Validate email addresses and other form inputs on the server side. Reject malformed input and make sure user-controlled values are safely handled before being passed to mail or other system functions.

Findings That Were Investigated but Not Confirmed

I also investigated possible WordPress and PHPMailer exploitation paths during the assessment. These were not added as confirmed vulnerabilities because I did not obtain enough evidence to prove successful exploitation.

The PHPMailer exploitation attempt did not create a session. Metasploit reported an attempted payload write to /var/www/2p2hOS7T.php, but requesting the corresponding web path returned HTTP 404.

I also tested the investigated WordPress ?static=1 behaviour, but it did not expose private or secret content.

Because these attacks were not successful, I have not listed them as confirmed vulnerabilities in the risk register.